

Fraud Detection in Financial Transactions

Final Project Report

Introduction to Artificial Intelligence - EARIN

Summer Semester 2026

Machine Learning Solution for Detecting Fraudulent Transactions

A complete supervised classification pipeline using the PaySim dataset, feature engineering, class imbalance handling, Random Forest modeling, visual analysis, and final performance evaluation.

Project Topic: Fraud Detection

Dataset: PaySim Synthetic Financial Dataset

Final Model: Random Forest Classifier

Prepared by: Tharun Pranav Senthil Kumar
Esin Kutlu

Instructor: Kowaleczko Paweł

June 9, 2026

Contents

1 Executive Summary 3

2 Introduction 3

3 Dataset Description 4

3.1 Dataset Statistics 4

3.2 Main Attributes 4

4 Exploratory Data Analysis 5

4.1 Class Imbalance 5

4.2 Transaction Type Analysis 6

4.3 Transaction Amount Distribution 7

5 Data Preprocessing and Feature Engineering 7

5.1 Engineered Features 7

5.2 Categorical Encoding 8

5.3 Dropped Columns 8

5.4 Train-Test Split 8

6 Implemented Models 8

6.1 Midterm Baseline: Logistic Regression 8

6.2 Final Model: Random Forest 8

7 Evaluation Metrics 9

8 Final Results 9

8.1 Final Metrics 10

8.2 Confusion Matrix 11

8.3 ROC Curve 12

8.4 Baseline Comparison 12

9	Feature Importance Analysis	13
10	Implementation Details	13
10.1	Libraries Used	13
10.2	Code Structure	14
10.3	How to Run the Project	14
11	Challenges and Solutions	15
12	Discussion	15
13	Conclusion	15
13.1	Future Work	16

1 Executive Summary

This project focuses on detecting fraudulent financial transactions using machine learning. The task is treated as a supervised binary classification problem where the model predicts whether a transaction is normal or fraudulent.

The PaySim dataset is highly imbalanced: only **8,213** transactions out of **6,362,620** are fraudulent, giving a fraud ratio of approximately **0.129%**. Because of this, accuracy alone is not enough to evaluate the model. The final evaluation uses precision, recall, F1-score, ROC-AUC, false positive rate, false negative rate, and confusion matrix analysis.

Final result: Random Forest was selected as the final model because it achieved very high recall and a strong ROC-AUC score while keeping the number of missed fraud cases very low. The final model detected **1,624 out of 1,643** fraud cases in the test set.

ROC-AUC
0.9993

Recall
98.84%

Missed Frauds
19

2 Introduction

Fraud detection is an important application of artificial intelligence in financial systems. Modern financial platforms process a very large number of transactions, so manual fraud checking is not practical. A machine learning model can help automatically identify suspicious transactions and support faster decision-making.

The goal of this project is to build a machine learning pipeline that predicts whether a transaction is fraudulent based on transaction details such as transaction type, amount, sender balance, receiver balance, and engineered balance-difference features.

The prediction task is defined as:

$$f(x) = \begin{cases} 1, & \text{fraudulent transaction} \\ 0, & \text{normal transaction} \end{cases}$$

The midterm phase included exploratory data analysis, preprocessing, feature engineering, Logistic Regression as a baseline model, and Random Forest as the primary model. In the final phase, the project was completed by cleaning the implementation, saving results and plots automatically, focusing on Random Forest as the final model, and improving the final documentation.

3 Dataset Description

The project uses the **PaySim Synthetic Financial Dataset**, available on Kaggle:

<https://www.kaggle.com/datasets/ealaxi/paysim1>

The dataset simulates mobile money transactions and contains both legitimate and fraudulent operations. It is useful for fraud detection research because it contains realistic transaction behavior while avoiding real customer-sensitive data.

3.1 Dataset Statistics

Table 1: Main dataset statistics

Property	Value
Total transactions	6,362,620
Fraudulent transactions	8,213
Normal transactions	6,354,407
Original number of features	11
Fraud ratio	0.129%

The dataset is strongly imbalanced. This is one of the main challenges of the project because a model could achieve high accuracy simply by predicting most transactions as normal. Therefore, the evaluation focuses more on fraud detection quality than on accuracy alone.

3.2 Main Attributes

Table 2: Selected dataset attributes

Attribute	Description
step	Time step of the transaction
type	Transaction type: PAYMENT, TRANSFER, CASH_OUT, DEBIT, CASH_IN
amount	Transaction amount
oldbalanceOrg	Sender balance before transaction
newbalanceOrig	Sender balance after transaction
oldbalanceDest	Receiver balance before transaction
newbalanceDest	Receiver balance after transaction
isFraud	Target variable: 1 for fraud, 0 for normal

4 Exploratory Data Analysis

The exploratory analysis was used to understand the dataset before model training. The most important findings were class imbalance, the importance of transaction type, and the wide range of transaction amounts.

4.1 Class Imbalance

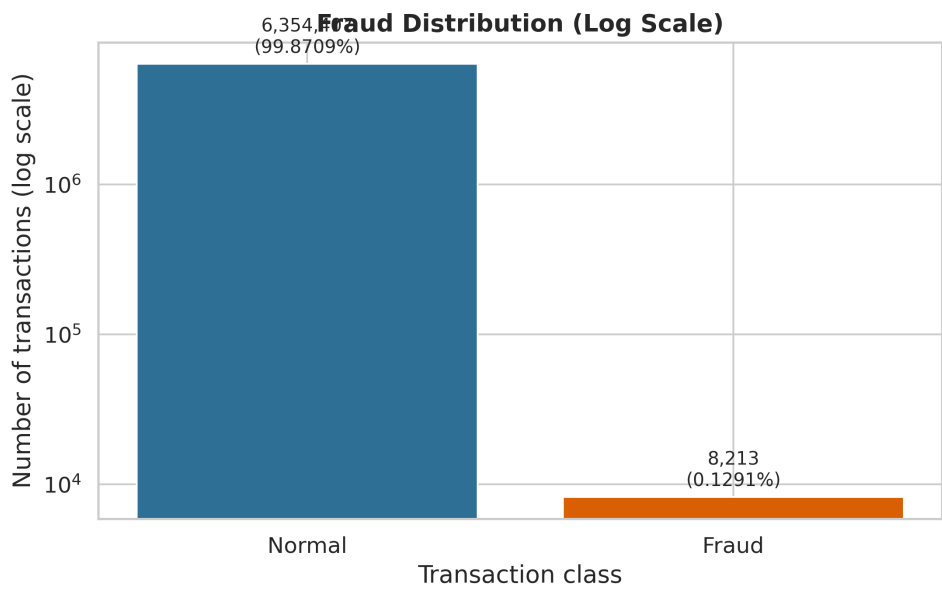


Figure 1: Fraud distribution shown on a logarithmic scale. The log scale makes the minority fraud class visible.

A normal count plot is not very useful here because fraud cases are extremely rare compared to normal transactions. For that reason, the final report uses a logarithmic scale. The plot clearly shows that fraud detection is an imbalanced classification problem.

4.2 Transaction Type Analysis

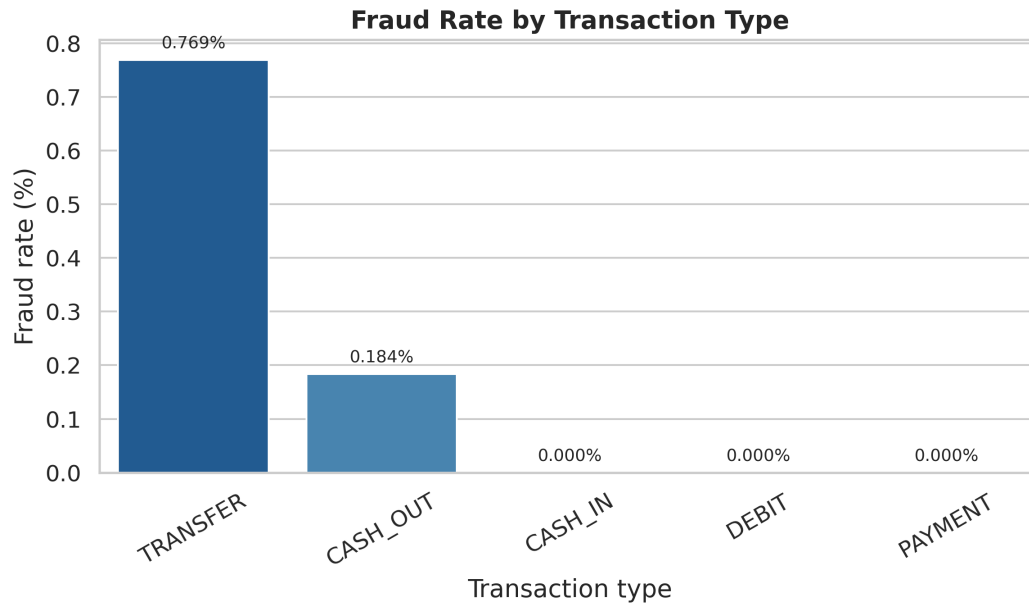


Figure 2: Fraud rate by transaction type. Fraud mainly appears in TRANSFER and CASH_OUT transactions.

The fraud rate is highest for TRANSFER transactions and lower for CASH_OUT. The remaining transaction types have no fraud cases in this dataset. This shows that transaction type is a useful signal for the model.

Table 3: Fraud percentage by transaction type

Transaction Type	Normal Percentage	Fraud Percentage
TRANSFER	99.2312%	0.7688%
CASH_OUT	99.8160%	0.1840%
CASH_IN	100.0000%	0.0000%
DEBIT	100.0000%	0.0000%
PAYMENT	100.0000%	0.0000%

4.3 Transaction Amount Distribution

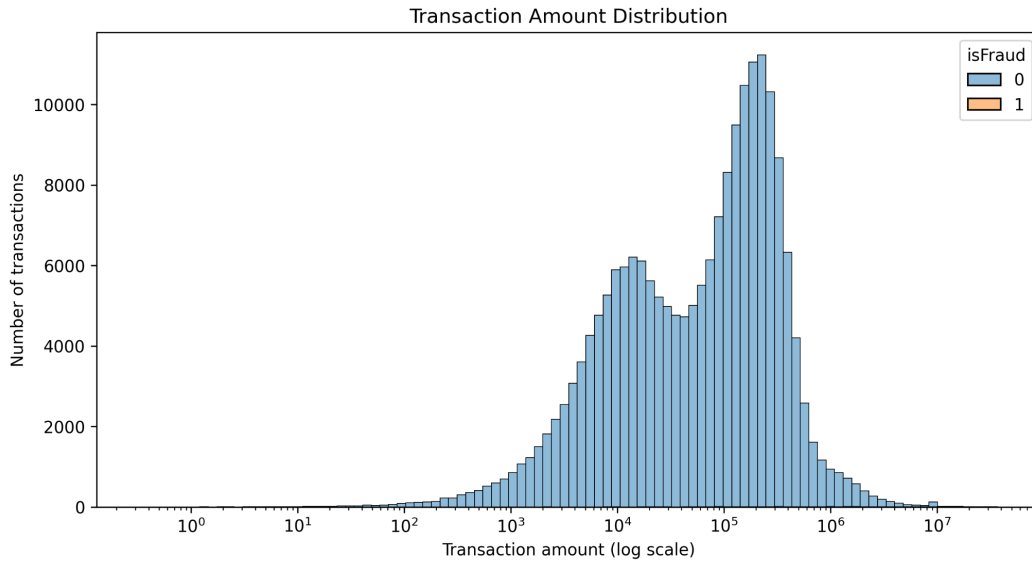


Figure 3: Distribution of transaction amounts on a logarithmic scale. The log scale is used because transaction amounts vary across a wide range.

The amount distribution shows that transactions cover multiple orders of magnitude. This supports the use of nonlinear models such as Random Forest because fraud patterns may depend on combinations of amount, balances, and transaction type.

5 Data Preprocessing and Feature Engineering

The final implementation uses a clean preprocessing pipeline. The goal is to prepare the raw dataset in a way that is suitable for Random Forest training while keeping the code simple and understandable.

5.1 Engineered Features

Two balance-difference features were added:

- **balanceOrigDiff**: sender balance change, computed as `oldbalanceOrig - newbalanceOrig`.
- **balanceDestDiff**: receiver balance change, computed as `newbalanceDest - oldbalanceDest`.

These features are useful because fraud patterns are often connected to abnormal balance changes. The final feature importance analysis confirms that **balanceOrigDiff** is the most important feature in the model.

5.2 Categorical Encoding

The transaction `type` column was converted using one-hot encoding. This creates separate binary columns such as `type_TRANSFER`, `type_CASH_OUT`, and `type_PAYMENT`. This allows the Random Forest model to learn patterns specific to each transaction type.

5.3 Dropped Columns

The following columns were removed:

- `nameOrig` and `nameDest`, because they are account identifiers and do not provide useful general predictive information.
- `isFlaggedFraud`, because it is a system-generated flag and may create leakage or make the model less general.

5.4 Train-Test Split

The dataset was split into **80% training data** and **20% testing data**. Stratified sampling was used to keep the fraud ratio similar in both sets. The random state was fixed at 42 to make the results reproducible.

6 Implemented Models

6.1 Midterm Baseline: Logistic Regression

During the midterm phase, Logistic Regression was implemented as a baseline model. It was useful because it is simple and easy to interpret. However, it produced many false positives compared with Random Forest. Therefore, it was not selected as the final model.

6.2 Final Model: Random Forest

Random Forest was selected as the final model because it can model nonlinear relationships and interactions between features. This is useful for fraud detection because fraudulent behavior may depend on several conditions at the same time, such as transaction type, amount, and balance changes.

Table 4: Random Forest configuration

Parameter	Value
n_estimators	100
max_depth	10
class_weight	balanced
random_state	42
n_jobs	-1

The `class_weight='balanced'` parameter was used to reduce the effect of the class imbalance problem. It gives higher importance to fraud cases during training.

7 Evaluation Metrics

Because the dataset is highly imbalanced, several metrics were used together. Accuracy is included, but it is not enough by itself.

Table 5: Evaluation metrics used in the final project

Metric	Purpose
Accuracy	Overall percentage of correct predictions
Precision	How many predicted fraud cases are actually fraud
Recall	How many real fraud cases were detected
Specificity	How many normal transactions were correctly classified
F1-score	Balance between precision and recall
ROC-AUC	Overall ability to separate fraud and normal transactions
False Positive Rate	Percentage of normal transactions wrongly marked as fraud
False Negative Rate	Percentage of fraud transactions missed by the model

For fraud detection, **recall** and **false negative rate** are especially important because missed fraud cases are costly. Precision is also important because too many false alarms can create extra manual investigation work.

8 Final Results

8.1 Final Metrics

Table 6: Final Random Forest performance on the test set

Metric	Value
Accuracy	98.9842%
Balanced Accuracy	98.9140%
Precision	11.1761%
Recall	98.8436%
Specificity	98.9844%
F1-score	20.0816%
ROC-AUC	0.9993
False Positive Rate	1.0156%
False Negative Rate	1.1564%
True Negatives	1,257,974
False Positives	12,907
False Negatives	19
True Positives	1,624

The Random Forest model correctly detected **1,624** fraudulent transactions and missed only **19**. This gives a high recall of **98.84%**. The precision is lower because the dataset is extremely imbalanced and the model still produces false positives. In a fraud detection system, this behavior can be acceptable when the priority is to avoid missing fraud.

8.2 Confusion Matrix

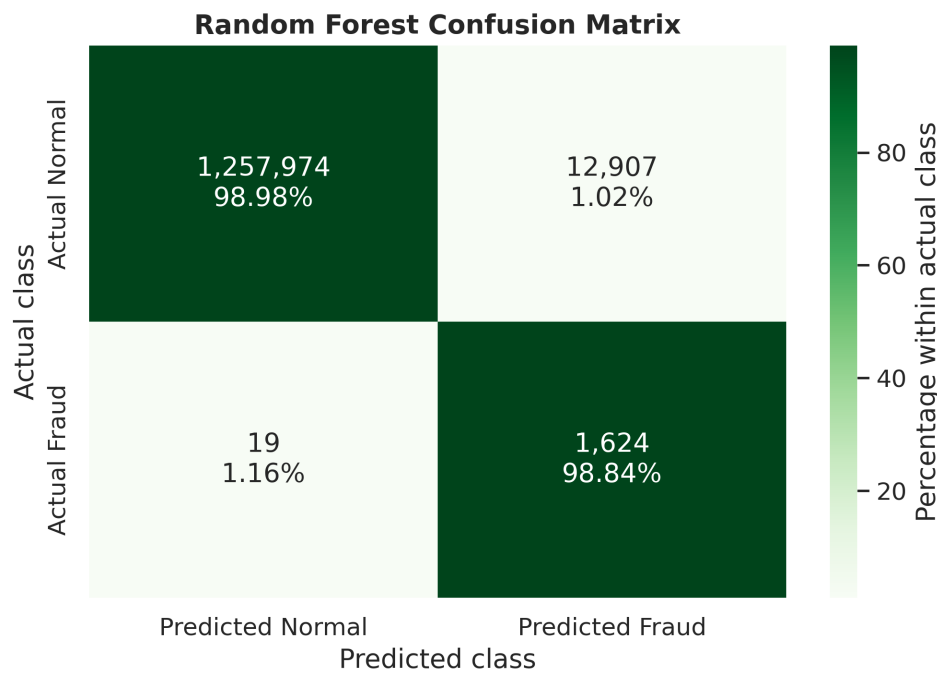


Figure 4: Random Forest confusion matrix with counts and row percentages.

The confusion matrix shows that the model performs well on both classes. For the fraud class, it correctly identifies **98.84%** of fraudulent transactions. For the normal class, it correctly classifies **98.98%** of transactions.

8.3 ROC Curve

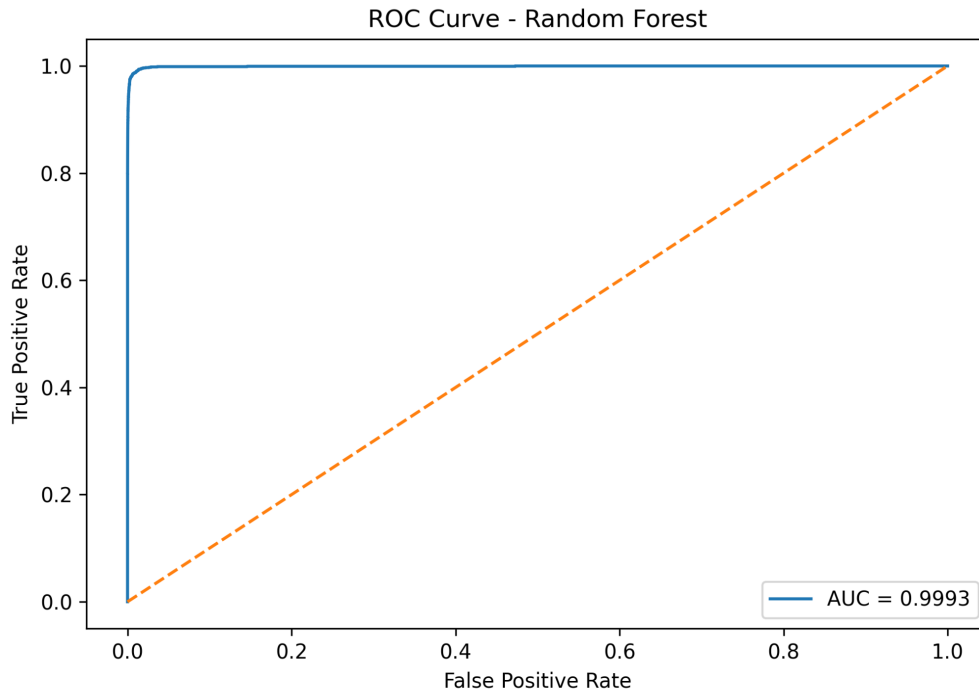


Figure 5: ROC curve for the Random Forest model with ROC-AUC = 0.9993.

The ROC curve is very close to the top-left corner, which means the model separates normal and fraudulent transactions very well.

8.4 Baseline Comparison

Table 7: Midterm baseline comparison

Metric	Logistic Regression	Random Forest
ROC-AUC	0.9908	0.9993
True Positives	1,579	1,624
False Positives	63,657	12,907
False Negatives	64	19
True Negatives	1,207,224	1,257,974

Random Forest improves the midterm baseline by detecting more fraud cases and greatly reducing false positives. This is why Random Forest was chosen as the final model.

9 Feature Importance Analysis

Random Forest provides feature importance scores. These scores show which features contributed most to the final prediction decisions.

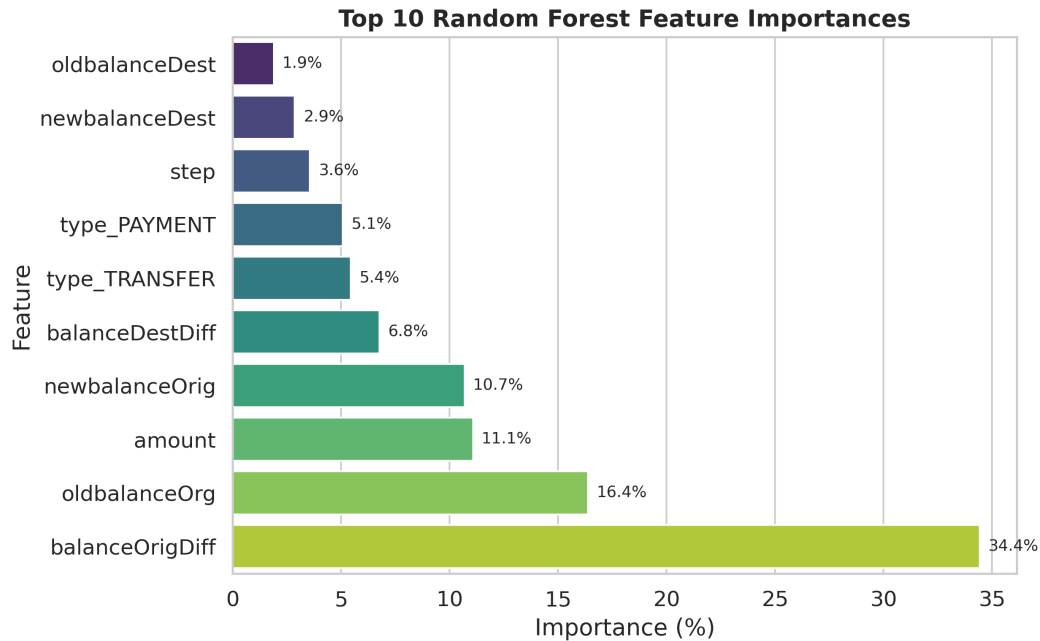


Figure 6: Top 10 feature importances of the Random Forest model.

The engineered feature `balanceOrigDiff` is the most important feature, contributing approximately **34.43%** of total feature importance. This confirms that the sender balance change is highly useful for identifying fraud. Other important features include `oldbalanceOrg`, `amount`, `newbalanceOrig`, and `balanceDestDiff`.

Key observation: The strongest signals are related to sender-side balance behavior. This supports the feature engineering approach and shows that fraud is not only related to transaction amount, but also to how account balances change before and after the transaction.

10 Implementation Details

The final project was implemented in Python using a modular script. The implementation was kept simple, readable, and student-friendly.

10.1 Libraries Used

- **pandas:** data loading and manipulation

- **numpy**: numerical operations
- **scikit-learn**: model training and evaluation
- **matplotlib** and **seaborn**: visualizations
- **joblib**: saving the trained model

10.2 Code Structure

The final Python file is organized into clear functions:

Table 8: Main functions in the final code

Function	Purpose
<code>load_dataset()</code>	Loads the PaySim CSV file
<code>print_basic_information()</code>	Prints dataset shape, missing values, and class counts
<code>create_eda_plots()</code>	Creates useful exploratory plots
<code>prepare_data()</code>	Performs feature engineering, encoding, and train-test split
<code>train_random_forest()</code>	Trains the final Random Forest model
<code>evaluate_random_forest()</code>	Calculates final model metrics and saves result tables
<code>create_evaluation_plots()</code>	Saves confusion matrix, ROC curve, precision-recall curve, and feature importance plot

10.3 How to Run the Project

To reproduce the results, the PaySim CSV file should be placed in the same folder as the Python script.

```
pip install -r requirements.txt
python Project_final.py
```

After running the script, the following outputs are created:

- **results/**: plots and CSV metric tables
- **models/random_forest_model.pkl**: saved trained Random Forest model

11 Challenges and Solutions

Table 9: Main challenges and how they were handled

Challenge	Solution
Severe class imbalance	Used <code>class_weight='balanced'</code> and evaluated the model using recall, F1-score, ROC-AUC, and confusion matrix instead of accuracy only
Fraud cases not visible in basic plots	Used logarithmic scale and fraud-rate visualizations
Large dataset size	Used efficient scikit-learn training and sampling only for some visualizations
Need for model explainability	Used Random Forest feature importance to identify key predictive features
Avoiding overcomplicated final solution	Kept Random Forest as the final model and focused on clean evaluation and documentation

12 Discussion

The final model achieves a very strong ROC-AUC score and very high recall. This means that the model is effective at identifying most fraud cases. The small number of false negatives is important because missed fraud transactions are usually more serious than false alerts.

At the same time, the precision is lower because the model predicts some normal transactions as fraud. This is expected in highly imbalanced fraud detection problems. In a real financial system, these false positives would usually be passed to a manual review process.

The feature importance results make the model more understandable. The most important features are related to sender balance changes, original sender balance, transaction amount, and receiver balance changes. This gives a clear explanation of what the model uses to identify suspicious behavior.

13 Conclusion

This project successfully implemented a complete fraud detection pipeline for the PaySim synthetic financial dataset. The work includes data exploration, preprocessing, feature engineering, class imbalance handling, model training, final evaluation, visualizations, and reproducible code organization.

The final Random Forest model achieved:

- ROC-AUC: 0.9993
- Recall: 98.84%
- Only 19 missed fraud cases out of 1,643 fraud cases in the test set
- Strong feature interpretability through Random Forest feature importance

Random Forest was selected as the final model because it provided the best practical balance between detecting fraud and reducing false positives. The final solution is simple, understandable, and suitable for presentation as a complete artificial intelligence project.

13.1 Future Work

Possible improvements include threshold tuning, more detailed precision-recall analysis, testing other tree-based models, and evaluating the model on other fraud datasets to check generalization.

References

- [1] PaySim Dataset,
<https://www.kaggle.com/datasets/ealaxi/paysim1>
- [2] Scikit-learn Documentation,
<https://scikit-learn.org/>
- [3] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.